

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 7 :

Preuves de corrections et de terminaison

Par:

Pr. Mohamed-Amine Chadi

Email: m.chadi@uca.ac.ma

Plan:

1. Introduction
 1. Motivation
 2. Types de preuves
2. Règles (ou étapes) de la preuve par induction
 1. Exemple 1 : somme des entiers positifs
 2. Exemple 2 : somme des carrés des entiers positifs
 3. Exemple 3 : propriété de Fibonacci
3. Règles (ou étapes) preuve par invariance de boucle
 1. Exemple 1 : fonction get_max
 2. Exemple 2 : tri par sélection
4. Preuve de terminaison
 1. Exemple 1 : recherche linéaire (à l'aide de boucle while)
 2. Exemple 2 : fonction récursive de countdown

Introduction

Motivation

- Lorsqu'on a vu les algorithmes tels que :
 - les algos de tri (bulles, insertion, sélection, fusion)
 - les algos de recherche (linéaire, dichotomique)
 - Etc.
- Nous les avons testés sur une liste (comme exemple), mais pas sur toutes les listes qui existent !!
- Du coup, la question est : comment peut-on être sûr que notre algorithme fonctionnera bien pour toutes les listes et dans toutes les situations
 - C'est exactement ce problème qu'adresse ce chapitre, sous le nom : **Preuves de correction**
- Les preuves de correction sont essentielles pour garantir que nos algorithmes produisent toujours les résultats attendus, quelque soient les entrées fournies.
- Elles nous aident à établir la confiance dans la fiabilité et la précision de nos programmes informatiques, ce qui est crucial dans de nombreux domaines, notamment la sécurité des systèmes, la santé, les finances et bien d'autres.
- Aussi, nous verrons que les preuves de correction peuvent nous aider à identifier d'autres formes d'un algorithme dont la complexité est significativement réduite

Types de preuves de correction

Ils existent deux types de preuves de correction

Preuve par induction

Utilisée pour les algorithmes qui utilisent une approche itérative, récursive, ou un calcul mathématique direct

Preuve par invariant de
boucle

Utilisée pour les algorithmes qui utilisent une approche itérative (des boucles for ou while). Parmi les étapes de cette méthode, on trouve la preuve par induction aussi

Preuve par induction

1. Cas de base :

- Montrer que c'est VRAI pour le cas le plus simple (comme $n=0$)

2. Cas inductif :

- Supposer que c'est VRAI pour $n=k$
- Montrer que c'est VRAI pour $n=k+1$

Exemple 1

Somme des entiers positifs

Somme des entiers positifs

$$\sum_{i=0}^n i = 0 + 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Etape 1 : cas de base (tester les cas les plus simples)

pour $n = 0$:

$$\sum_{i=0}^{n=0} i = 0 = \frac{0(0+1)}{2} \quad \text{True}$$

pour $n = 1$:

$$\sum_{i=0}^{n=1} i = 0 + 1 = \frac{1(1+1)}{2} = 1 \quad \text{True}$$

pour $n = 2$:

$$\sum_{i=0}^{n=2} i = 0 + 1 + 2 = \frac{2(2+1)}{2} = 3 \quad \text{True}$$

pour $n = 3$:

$$\sum_{i=0}^{n=3} i = 0 + 1 + 2 + 3 = \frac{3(3+1)}{2} = 6 \quad \text{True}$$

$$\sum_{i=0}^n i = 0 + 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Etape 2 : supposer que c'est VRAI pour n=k

pour n = k :

$$\sum_{i=0}^{n=k} i = 0 + 1 + 2 + 3 + \dots + k = \frac{k(k+1)}{2}$$

$$\sum_{i=0}^n i = 0 + 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Etape 3 : montrer que c'est VRAI pour $n=k+1$

pour $n = k+1$:

$$\sum_{i=0}^{n=k+1} i = \underbrace{0 + 1 + 2 + 3 + \dots + k}_{\frac{k(k+1)}{2}} + \underbrace{(k+1)}_{(k+1)} = \frac{(k+1)[(k+1)+1]}{2} = \frac{(k+1)(k+2)}{2}$$

$$\frac{k(k+1)}{2} + (k+1)$$

$$= \frac{k(k+1) + 2(k+1)}{2}$$

$$= \frac{(k+1) + (k+2)}{2}$$

Donc, terme gauche égale au terme droite $\rightarrow$ algorithme correcte

Quel est le bénéfice de changer code 1 par code 2 ?

$$\sum_{i=0}^n i = 0 + 1 + 2 + 3 + \dots + n$$

```
def somme_nombres_naturels(n):  
    s = 0  
    for i in range(1, n+1):  
        s += i  
  
    return s
```

```
n = 5 #0  
somme_nombres_naturels(n)
```

15

$$\frac{n(n+1)}{2}$$

```
def somme_nombres_naturels(n):  
    return (n * (n + 1)) // 2
```

```
n = 5  
somme_nombres_naturels(n)
```

15

Complexité du 1^{er} code est O(n)
Complexité du 2^{ème} code est O(1)

Exemple 2

Somme des carrés des entiers positifs

Somme des carrés des entiers positifs

De même,
imaginez que vous devez
calculer cette série pour $n =$
1milliard,
vous allez donc faire
1milliard d'itérations,
Alors qu'avec la formule
« analytique », la complexité
temporelle est juste de $O(1)$,
avec 5 opérations
élémentaires

$$\sum_{i=0}^n i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

- Prouver que c'est correct
- et donner la fonction python correspondant à chaque terme (gauche et droite)

Etape 1 : cas de base (tester les cas les plus simples)

pour $n = 0$:

$$\sum_{i=0}^{n=0} i^2 = 0^2 = \frac{0(0+1)(2*0+1)}{6} = 0$$

True

pour $n = 1$:

$$\sum_{i=0}^{n=1} i^2 = 0^2 + 1^2 = \frac{1(1+1)(2*1+1)}{6} = 1$$

True

$$\sum_{i=0}^n i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Etape 2 : supposer que c'est VRAI pour n=k

pour n = k :

$$\sum_{i=0}^{n=k} i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + k^2 = \frac{k(k+1)(2k+1)}{6}$$

$$\sum_{i=0}^n i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Etape 3 : montrer que c'est VRAI pour n=k+1

pour n = k+1 :

$$\sum_{i=0}^{n=k+1} i^2 = \underbrace{0^2 + 1^2 + 2^2 + 3^2 + \dots + k^2}_{\frac{k(k+1)(2k+1)}{6}} + \underbrace{(k+1)^2}_{(k+1)^2} = \frac{(k+1)(k+2)(2(k+1)+1)}{6}$$

$$\frac{k(k+1)(2k+1)}{6} + (k+1)^2$$

$$= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} = \frac{(k^2+k)(2k+1) + 6(k^2+2k+1)}{6}$$

$$= \frac{2k^3 + k^2 + 2k^2 + k + 6k^2 + 12k + 6}{6} = \frac{2k^3 + 9k^2 + 13k + 6}{6}$$

Nous avons maintenant le résultat du terme gauche
Allons-y pour voir le terme droit

$$\sum_{i=0}^n i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Etape 3 : montrer que c'est VRAI pour n=k+1

pour n = k+1 :

$$\sum_{i=0}^{n=k+1} i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + k^2 + (k+1)^2 = \underbrace{\frac{(k+1)(k+2)(2(k+1)+1)}{6}}$$

$$\begin{aligned} \frac{(k+1)(k+2)(2(k+1)+1)}{6} &= \frac{(k^2 + 2k + k + 2)(2k + 2 + 1)}{6} \\ &= \frac{(k^2 + 3k + 2)(2k + 3)}{6} = \frac{2k^3 + 3k^2 + 6k^2 + 9k + 4k + 6}{6} \\ &= \frac{2k^3 + 9k^2 + 13k + 6}{6} \end{aligned}$$

Donc, terme gauche égale au terme droite → algorithme correcte

On peut donc remplacer le code itératif avec un simple code de complexité $O(1)$

$$\sum_{i=0}^n i^2 = 0^2 + 1^2 + 2^2 + 3^2 + \dots + n^2$$

$$\frac{n(n+1)(2n+1)}{6}$$

```
def somme_carre(n):  
    sc = 0  
    for i in range(1, n+1):  
        sc += i**2  
  
    return sc  
  
somme_carre(5)
```

55

```
def somme_carre(n):  
    sc = (n*(n+1)*(2*n+1)) / 6  
    return sc
```

```
somme_carre(5)
```

55.0

Complexité du 1^{er} code est $O(n)$
Complexité du 2^{ème} code est $O(1)$

Exemple 3

Propriété de la suite de Fibonacci

Propriété de la suite de Fibonacci

$$f_{n+2} = f_n + f_{n-1} + f_{n-2} + \dots + 1$$

Tout d'abord, on sait que :

$$f_{n+1} = f_n + f_{n-1} \quad (\text{définition de la suite de Fibonacci})$$

$$0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 + \dots$$

Etape 1 : cas de base (tester les cas les plus simples)

pour $n = 0$:

$$\text{terme de gauche} = f_{n+2} = f_{0+2} = f_2 = 1$$

$$\text{terme de droite} = f_0 + 1 = 0 + 1 = 1$$

terme de gauche = terme de droite $\rightarrow$ correcte pour $n=0$

pour $n = 1$:

$$\text{terme de gauche} = f_{n+2} = f_{1+2} = f_3 = 2$$

$$\text{terme de droite} = f_1 + 1 = 1 + 1 = 2$$

terme de gauche = terme de droite $\rightarrow$ correcte pour $n=1$

$$f_{n+2} = f_n + f_{n-1} + f_{n-2} + \cdots + 1$$

Etape 2 : supposer que c'est VRAI pour n=k

pour n = k :

$$f_{k+2} = f_k + f_{k-1} + f_{k-2} + \cdots + 1$$

$$f_{n+2} = f_n + f_{n-1} + f_{n-2} + \cdots + 1$$

Etape 3 : montrer que c'est VRAI pour $n=k+1$

pour $n = k+1$:

$$f_{k+3} = f_{k+1} + f_k + f_{k-1} + \cdots + 1$$

On commence depuis le terme de gauche,

On sait depuis la définition de la suite de Fibonacci que :

$$f_{n+1} = f_n + f_{n-1}, \text{ donc}$$

$$f_{k+3} = f_{k+2} + f_{k+1}$$

On sait depuis notre supposition dans étape 2 que :

$$f_{k+2} = f_k + f_{k-1} + f_{k-2} + \cdots + 1, \text{ donc}$$

$$f_{k+3} = f_{k+2} + f_{k+1}$$

$$f_{k+3} = f_k + f_{k-1} + f_{k-2} + \cdots + 1 + f_{k+1} = \text{terme de droite}$$

Preuve par invariant de boucle

1. **Identifier l'invariant** : Un invariant de boucle est une propriété sur laquelle elle se base l'algorithme pour bien fonctionner et doit rester vraie avant, pendant et après chaque itération de la boucle. C'est une condition qui ne change pas et qui garantit que l'algorithme fonctionne correctement à chaque étape.
2. **Initialisation** : Montrer que l'invariant est VRAI pour la première itération
3. **Conservation** : Supposer que c'est VRAI pour $i = k$, et montrer que c'est VRAI pour $i = k+1$ (par induction)
4. **Terminaison** : Montrer que c'est VRAI pour la dernière itération où il termine

get_max

Etape 1 : définir l'invariant de boucle

on doit démontrer l'invariant suivant :

pour chaque itération i ,

m est toujours le max de la liste $a[0 : i]$

Etape 2 : initiation

montrer que c'est vrai lorsqu'on finit la première itération

pour $i = 1$

$a[0 : i]$ contient un seul élément ($a[0]$)

bien sûr, un seul élément, c'est aussi le max

Etape 3 : conservation

supposer que c'est vrai pour $i = k$,

c.à.d., m est le max de la liste $a[0 : k]$

et montrer que c'est vrai pour $i = k+1$

pour $i = k+1$,

l'algorithme compare m précédent avec $a[k+1]$

si $a[k+1]$ est $> m$: change la valeur de m par $a[k+1]$,

par conséquent, m est toujours le max de la liste $a[0 : k+1]$

Etape 4 : Terminaison

pour $i = \text{len}(a) - 1$

m est bien évidemment le max de la liste $a[0 : \text{len}(a)]$

```
def get_max(a):  
    m = a[0]  
    for i in range(1, len(a)):  
        if a[i] > m:  
            m = a[i]
```

```
    return m
```

```
a = [2, 8, 9, 1, 7]  
get_max(a)
```

9

Tri par sélection

```
def tri_selection(arr):
    n = len(arr)

    for i in range(n - 1):
        # Trouver l'élément minimum dans la partie non triée du tableau
        indice_min = i
        for j in range(i + 1, n):
            if arr[j] < arr[indice_min]:
                indice_min = j

        # Échanger l'élément minimum trouvé avec le premier élément
        arr[i], arr[indice_min] = arr[indice_min], arr[i]

    return arr

a = [10, 7, 17, 25, 2]
a_trié = tri_selection(a)
print(a_trié)
```

```
[2, 7, 10, 17, 25]
```

Etape 1 : définir l'invariant de boucle

On doit démontrer l'invariant suivant :

à la fin de chaque itération i ,
le tableau $a[0 : i]$ est trié, et les éléments de $a[i : \text{len}(a)]$
sont $\geq a[i]$

Etape 2 : initiation

montrer que c'est vrai pour la première itération

$a[0 : i]$ contient un seul élément ($a[0]$)

par conséquent :

- le tableau est trié
- Lorsque $i=0$ termine, $a[0]$ sera le min de toute la liste, tous les éléments après 'i' sont $\geq a[0]$

Tri par sélection

```
def tri_selection(arr):
    n = len(arr)

    for i in range(n - 1):
        # Trouver l'élément minimum dans la partie non triée du tableau
        indice_min = i
        for j in range(i + 1, n):
            if arr[j] < arr[indice_min]:
                indice_min = j

        # Échanger l'élément minimum trouvé avec le premier élément
        arr[i], arr[indice_min] = arr[indice_min], arr[i]

    return arr

a = [10, 7, 17, 25, 2]
a_trié = tri_selection(a)
print(a_trié)

[2, 7, 10, 17, 25]
```

Etape 4 : Terminaison

montrer que c'est vrai lorsqu'on finit la dernière itération

pour $i = n-1$

- $a[0 : n]$ est trié
- Et $a[n-1]$ est le dernier élément de la liste, il n'y a un élément après avec quoi le comparer

→ Algorithme correcte

Etape 3 : conservation

supposer que c'est vrai pour $i = k$, c.à.d. :

- $a[0 : k]$ est trié
- Les éléments de $a[k:\text{len}(a)]$ sont $\geq a[k]$

et montrer que c'est vrai pour $i = k+1$

pour $i = k+1$, c.à.d.,

- le tableau $a[0 : k+1]$ est trié,
- et les éléments de $a[k+1 : \text{len}(a)]$ sont $\geq a[k+1]$

- dans l'itération $i = k+1$, on prend le min de la partie $a[k+1 : \text{len}(a)]$ et on le place dans $a[k+1]$, sans toucher à la partie $a[0:k]$

- et on sait que $a[k+1]$ est $\geq a[k]$ (depuis la supposition)

- donc :

- le tableau $a[0 : k+1]$ est trié
- Et parce que $a[k+1]$ est le min de la liste $a[k+1:\text{len}(a)]$, tous les éléments après $k+1$ sont supérieur ou égal à $a[k+1]$

Preuve de terminaison : par variant de boucle

Preuve de terminaison : variant de boucle

La preuve par variant de boucle se base sur 3 piliers :

1. Toute algorithmes sans appel de fonction ni structure répétitive termine
2. Toute répétitive «for» itère un nombre fini de fois
3. Une répétitive «while» (et les appels récursifs) peut boucler à l'infini si sa condition d'arrêt n'est jamais validée.

Donc, pour prouver la terminaison d'un algorithme il faudra montrer que les répétitives while ne bouclent pas à l'infini, c'est-à-dire que leurs conditions deviennent fausses au bout d'un certain nombre d'itération, et/ou que les appels récursives atteignent leurs cas de base.

Pour établir la terminaison d'un programme, on doit :

1. **Identifier le variant** : **Un variant** est une expression à valeur entière dépendant des variables impliquées dans la structure répétitive dont on peut démontrer la :
2. **Positivité ou nullité du variant** : positive ou nulle (du fait des initialisations et de la condition de la répétitive). Si le variant est initialisé à une valeur négative et que la boucle dépend de ce variant pour se terminer lorsque le variant devient nul, la boucle ne se terminera jamais car le variant ne pourra jamais devenir nul s'il est initialement négatif.
3. **Décroissance du variant** : décroît strictement au cours des itérations
4. **Atteinte de la condition d'arrêt** : le processus de décroissance utilisé amènera le variant à la valeur de la condition d'arrêt.

```
def appartient(v, T):  
    i = 0  
    trouvee = False  
    while i < len(T) and trouvee == False:  
        if T[i] == v:  
            trouvee = True  
        i = i + 1  
    return trouvee  
  
a = [10, 7, 2, 8, 19, 21, 3]  
appartient(19, a)
```

True

Exemple de la recherche linéaire

Il s'agit de trouver un variant de la boucle while, c'est-à-dire une quantité entière qui diminue strictement à chaque itération tout en restant positive.

Un variant possible est : $\text{len}(T) - i$, en effet, $\text{len}(T) - i$ est un entier puisque c'est la différence de deux entiers :

1. **Positivité ou nullité du variant** : Avant la première itération : $\text{len}(T) - i = \text{len}(T) \geq 0$.
2. **Décroissance du variant** : A chaque itération : i augmente de 1 (par l'instruction $i = i + 1$) et $\text{len}(T)$ est reste fixe, donc $\text{len}(T) - i$ diminue de 1.
3. **Atteinte de la condition d'arrêt** : le processus de décroissance ($i = i+1$) amènera le variant « $\text{len}(T)-i$ » à la condition d'arrêt $\text{len}(T)-i = 0$.

On vient de prouver que la boucle while termine et donc cet algorithme termine.

Exemple d'une fonction récursive

```
def countdown(n):  
    if n == 0:  
        print("Terminé!")  
    else:  
        print(n)  
        countdown(n-1)  
  
countdown(5)
```

Le variant ici c'est n lui-même, puisque:

- n est un entier positif,
- n diminue par 1 à chaque itération,
- la condition d'arrêt est que $n=0$.

Mais attention ici, si on donne un 'n' négatif, ce programme donc ne termine pas, du coup on doit rajouter une condition sur la valeur de n entrée par l'utilisateur.

► *## si n donnée par l'utilisateur est négative, la fct ne termine pas*

```
def countdown(n):  
    if n == 0:  
        print("Terminé!")  
    else:  
        print(n)  
        countdown(n-1)
```

```
countdown(-1)
```

```
-2967  
-2968  
-2969
```

RecursionError

Traceback (most recent call last)

Cell In[1], line 10

```
7         print(n)  
8         countdown(n-1)  
--> 10 countdown(-1)
```

Solution

on doit donc ajouter une condition sur la valeur de n

```
def countdown(n):  
    if n < 0:  
        print("n doit être un entier positif")  
    elif n == 0:  
        print("Terminé!")  
    else:  
        print(n)  
        countdown(n-1)
```

```
countdown(-1)
```

n doit être un entier positif

Exemple boucle while qui ne termine pas

```
i = 1
while i != 10:
    print(i)
    i += 2
```

Le variant ici est $10 - i$:

- $10 - i$ est un entier positif,
- $10 - i$ diminue par 2 via le processus ($i += 2$)
- Mais, la condition d'arrêt « $10 - i = 0$ » ne sera pas atteinte puisque i commence de 1, et augmentant 1 par 2 dans chaque itération donnera que des valeurs impaires qui ne coïncideront jamais avec 0.

Donc , programme ne termine pas